```
$loader = new \Twig\Loader\FilesystemLoader(__DIR__ . '/views');
$twig = new \Twig\Environment($loader);

$func = new \Twig\TwigFunction('add_humans', function ($string) {
    return $string . " is in charge. Anna and Paul are her slaves.";
});
$twig->addFunction($func);

print $twig->render('add-humans.html.twig', [
    'cat' => 'Jess'
]);
```

```
{# views/add-humans.html.twig #}
<div>
  {% if cat == 'Jess' %}
    {% set cat = add_humans(cat) %}
  {% endif %}
  <p>{{ cat }}</p>
</div>
```

```
$ curl http://localhost:8080/index.php
<div>
  <p>Jess is in charge. Anna and Paul are her slaves.</p>
</div
```

The example above is self-explanatory and is not a particularly good use of a function: it would be better implemented as a filter. Tread carefully when introducing functions (or any other extension to Twig); the more logic and data fetching/transformation that is involved, the more you are probably straying from the path of MVC and setting up potential problems in the future. Twig, or any other modern templating library, is not frictionless – far from it – but remember that the friction is there for a reason: to segregate content generation from other tasks that are better implemented elsewhere.